

Предисловие

С SQL все максимально знакомо, тут нейронки не использовал для решения.

Кроме 8 задания. С рекурсией ни разу не сталкивался, но в итоге разобрал механику, комментарии больше для себя оставил.

Задача №1.

Задача: Напишите запрос, объединяющий две таблицы `orders` и `customers`, возвращающий имена клиентов (`customer_name`) и сумму заказов (`order_total`), при условии, что заказ сделан клиентом старше 30 лет.

Таблицы:

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    age INT  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_total DECIMAL(10, 2)  
);
```

Решение

The screenshot shows a SQL IDE interface. On the left, a tree view displays the database structure: Clickhouse (localhost:8123) and learn_base (localhost:5432). Under learn_base, there are schemas (kafka_farm, nyc_taxi, public, steam) and a database test_sber. Inside test_sber, there are tables customers (56K), orders (24K), and users_orders (8K). The main editor shows a SQL query for 'Задание 1':

```
-- Задание 1  
SELECT  
    c.customer_name AS "Имя клиента",  
    SUM(o.order_total) AS "Сумма заказов"  
FROM  
    test_sber.customers AS c  
INNER JOIN  
    test_sber.orders AS o  
    ON c.customer_id = o.customer_id  
    AND c.age > 30  
GROUP BY c.customer_name
```

Below the query editor, the results are displayed in a table titled 'customers 1'. The table has two columns: 'Имя клиента' and 'Сумма заказов'. The results are sorted by the sum of orders in descending order.

	Имя клиента	Сумма заказов
1	Михаил Соколов	1,234.5
2	Мария Иванова	6,240.65
3	Ольга Новикова	5,795.8
4	Дмитрий Морозов	13,570.8
5	Алексей Смирнов	13,560.25
6	Сергей Сидоров	6,860.3

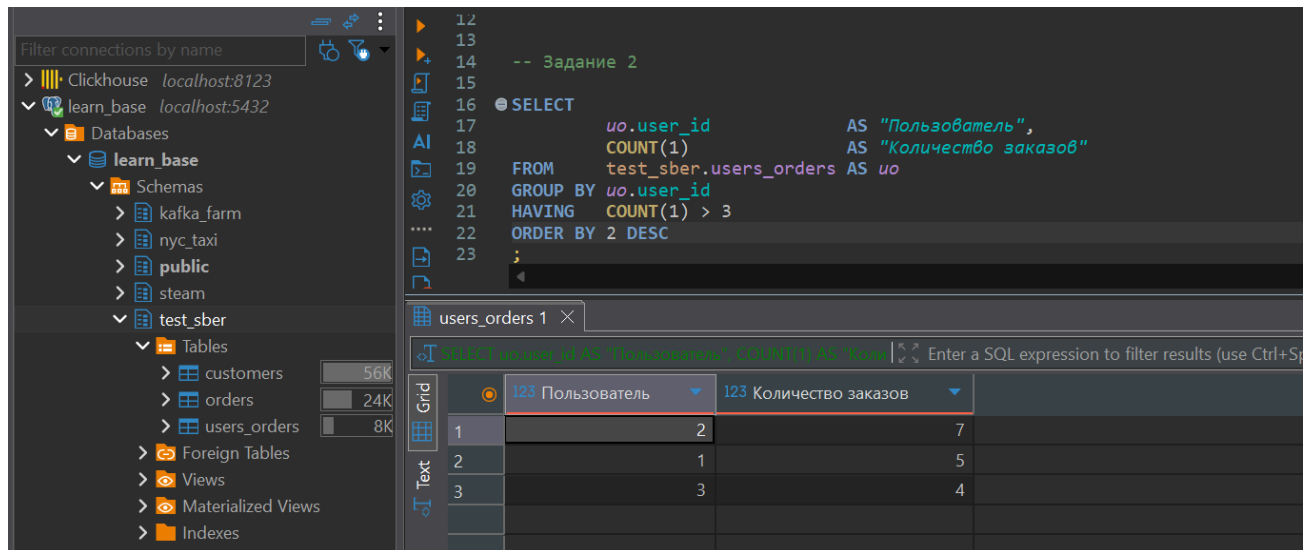
Задача №2.

Задача: Найдите всех пользователей, которые сделали больше трех заказов суммарно, отсортировав результат по количеству заказов в порядке убывания.

Таблица:

```
CREATE TABLE users_orders (
    user_id INT,
    order_date DATE,
    amount DECIMAL(10, 2)
);
```

Решение



The screenshot shows a database IDE with a SQL query editor and a results grid. The query is as follows:

```
-- Задание 2
SELECT
    uo.user_id AS "Пользователь",
    COUNT(1) AS "Количество заказов"
FROM test_sber.users_orders AS uo
GROUP BY uo.user_id
HAVING COUNT(1) > 3
ORDER BY 2 DESC
```

The results grid shows the following data:

	123 Пользователь	123 Количество заказов
1	2	7
2	1	5
3	3	4

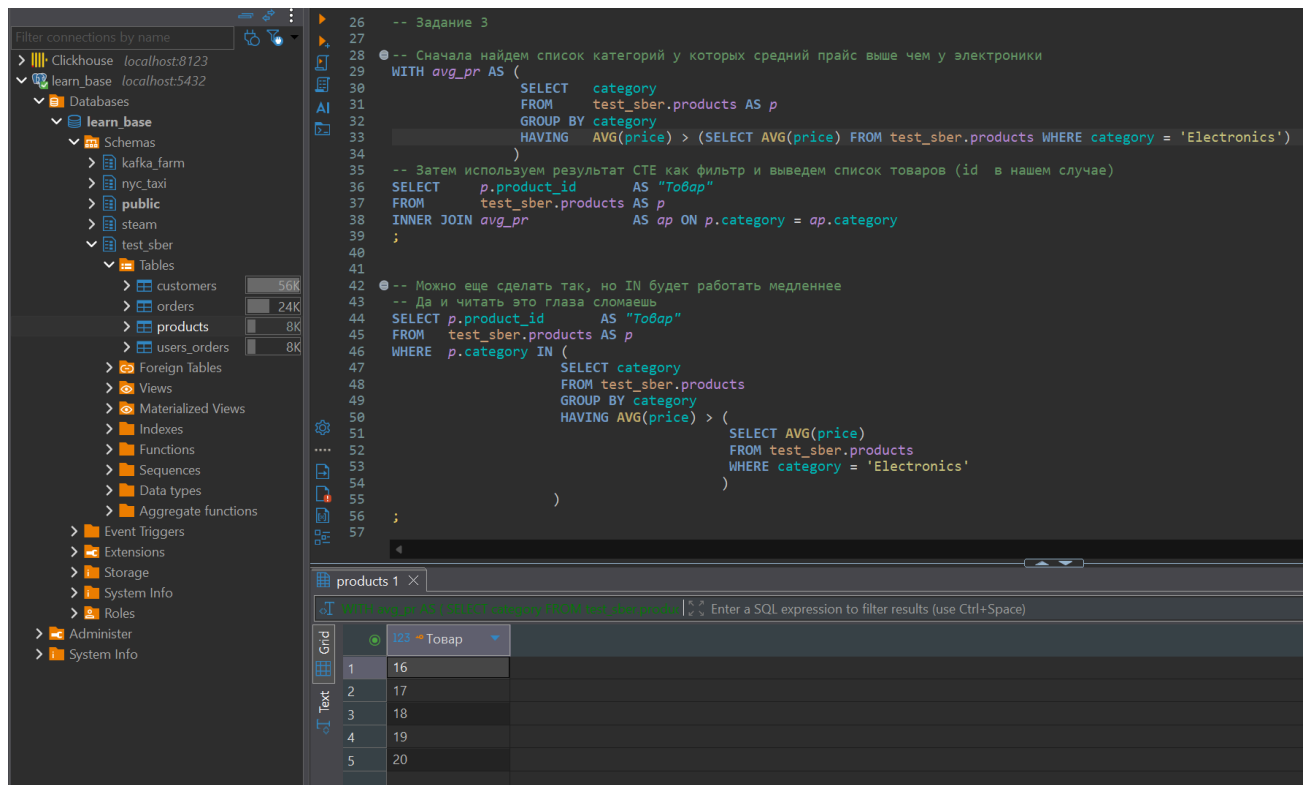
Задача №3.

Задача: Получите список товаров, средняя цена которых выше средней цены всех товаров категории 'Electronics'.

Таблицы:

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    category VARCHAR(50),
    price DECIMAL(10, 2)
);
```

Решение



Задача №4.

Задача: Верните таблицу сотрудников с указанием среднего заработка сотрудника относительно отдела, сгруппированного по отделам.

Таблицы:

```

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);

```

Решение

The screenshot shows the ClickHouse web interface. On the left is a sidebar with a tree view of databases and tables. The main panel displays a SQL query and its results.

SQL Query:

```

-- Задание 4
-- Тут насколько я понял нужен именно список сотрудников, а не просто средняя ЗП в отделе,
-- соответственно юзаем оконные функции
SELECT
    employee_id AS "Сотрудник",
    department AS "Отдел",
    AVG(salary) OVER(PARTITION BY department) AS "Средняя ЗП в данном отделе"
FROM test_sber.employees

```

Results Table:

	Сотрудник	Отдел	Средняя ЗП в данном отделе
1	19	Finance	77,000
2	18	Finance	77,000
3	20	Finance	77,000
4	17	HR	58,666.6666666667
5	16	HR	58,666.6666666667
6	15	HR	58,666.6666666667
7	1	IT	81,800
8	2	IT	81,800
9	3	IT	81,800
10	4	IT	81,800
11	5	IT	81,800
12	12	Marketing	69,000
13	13	Marketing	69,000
14	14	Marketing	69,000
15	11	Marketing	69,000
16	10	Sales	69,000
17	9	Sales	69,000
18	8	Sales	69,000
19	7	Sales	69,000
20	6	Sales	69,000

Задача №5.

Задача: Создать отчет, классифицирующий товары по категориям цен ('Дешевые', 'Средняя стоимость', 'Дорогие'), исходя из следующего условия:

- Если цена меньше 10 долларов — 'Дешевые'
- От 10 до 50 долларов включительно — 'Средняя стоимость'
- Больше 50 долларов — 'Дорогие'

Таблица:

```

CREATE TABLE goods (
    good_id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10, 2)
);

```

Решение

```

73
74 -- Задание 5
75
76 ● SELECT
77     good_id,
78     name,
79     price,
80     CASE
81         WHEN price < 10 THEN 'Дешевые'
82         WHEN price >= 10 AND price <= 50 THEN 'Средняя стоимость'
83         WHEN price > 50 THEN 'Дорогие'
84         -- больше 50 лучше прописать явно вместо ELSE,
85         -- иначе можем классифицировать NULL значения как "Дорогие", что некорректно
86     END AS "category"
87 FROM test_sber.goods
88
89

```

	123 good_id	AZ name	123 price	AZ category
1	1	Ручка шариковая	1.5	Дешевые
2	2	Тетрадь общая	3.25	Дешевые
3	3	Карандаш простой	0.99	Дешевые
4	4	Ластик	0.75	Дешевые
5	5	Линейка пластиковая	2.3	Дешевые
6	6	Калькулятор простой	15	Средняя стоимость
7	7	Набор фломастеров	25.5	Средняя стоимость
8	8	Ежедневник	35	Средняя стоимость
9	9	Сумка для обуви	42.75	Средняя стоимость
10	10	Набор красок	50	Средняя стоимость
11	11	Рюкзак кожаный	85	Дорогие
12	12	Графический планшет	150	Дорогие
13	13	Ноутбук	750	Дорогие
14	14	Электронная книга	120	Дорогие
15	15	Профессиональный калькулятор	75.5	Дорогие
16	16	Ручка пустая	[NULL]	[NULL]

Задача №6. Объединение временных интервалов (DATE DIFFERENCE)

Задача: Вычислить разницу в днях между датой регистрации клиента и сегодняшним днем.

Таблица:

```

CREATE TABLE clients (
    client_id INT PRIMARY KEY,
    registration_date DATE
);

```

Решение

```

89
90 -- Задание 6
91
92 ● SELECT
93     client_id AS "Клиент",
94     CURRENT_DATE AS "Сегодня",
95     registration_date AS "Дата регистрации",
96     -- Для Postgres можно просто вычесть, в MSSQL отдельная функция
97     CURRENT_DATE - registration_date AS "Разница в днях"
98 FROM test_sber.clients
99
100
101

```

	123 Клиент	Сегодня	Дата регистрации	123 Разница в днях
1	1	2026-03-15	2026-03-15	0
2	2	2026-03-15	2026-03-14	1
3	3	2026-03-15	2026-03-08	7
4	4	2026-03-15	2026-03-10	5
5	5	2026-03-15	2026-02-15	28
6	6	2026-03-15	2026-02-18	25
7	7	2026-03-15	2025-12-15	90
8	8	2026-03-15	2026-01-29	45
9	9	2026-03-15	2025-03-15	365
10	10	2026-03-15	2025-08-27	200
11	11	2026-03-15	2024-01-15	790
12	12	2026-03-15	2023-12-10	826
13	13	2026-03-15	2023-06-20	999
14	14	2026-03-15	2022-11-05	1,226
15	15	2026-03-15	2021-03-25	1,816

Задача №7.

Задача: Определить рост продаж каждого товара по сравнению с предыдущим месяцем.

Таблица:

```
CREATE TABLE sales (  
    product_id INT,  
    sale_month DATE,  
    revenue DECIMAL(10, 2)  
);
```

Решение

Filter connections by name

Clickhouse localhost:8123

learn_base localhost:5432

Databases

learn_base

Schemas

kafka_farm

nyc_taxi

public

steam

test_sber

Tables

clients 24K

customers 56K

employees 24K

goods 24K

orders 8K

products 24K

sales 8K

users 8K

users_orders 8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

Administrator

System Info

101 -- Задание 7

102

103

104 -- Тут распыленная формулировка, может трактоваться по разному

105 -- Мы ищем именно факт роста, процентное изменение, или абсолютное изменение продаж

106 -- Тут прям напрашивается уточнение T3

107 -- В общем сделал так и так

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

SELECT

product_id,

sale_month,

revenue,

LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) AS "revenue_lag1",

revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) AS "revenue_change_lag1",

(revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month)) /

LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) * 100 AS "percent",

CASE

WHEN

LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) IS NULL THEN 'Нет данных'

WHEN

revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) > 0

THEN 'Рост'

WHEN

revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) = 0

THEN 'Плато'

WHEN

revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) < 0

THEN 'Падение'

AS "grow_status"

END

FROM test_sber.sales

ORDER BY 1,2

;

sales 1

SELECT product_id, sale_month, revenue, LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) AS revenue_lag1, (revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month)) / LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) * 100 AS percent, CASE WHEN LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) IS NULL THEN 'Нет данных' WHEN revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) > 0 THEN 'Рост' WHEN revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) = 0 THEN 'Плато' WHEN revenue - LAG(revenue, 1) OVER(PARTITION BY product_id ORDER BY sale_month) < 0 THEN 'Падение' AS grow_status

Grid

1

2

3

4

5

6

7

8

9

10

11

product_id

sale_month

revenue

revenue_lag1

revenue_change_lag1

percent

grow_status

1

2024-01-01

1,000

[NULL]

[NULL]

[NULL]

Нет данных

2

2024-02-01

1,200

1,000

200

20

Рост

3

2024-03-01

1,500

1,200

300

25

Рост

4

2024-01-01

2,000

[NULL]

[NULL]

[NULL]

Нет данных

5

2024-02-01

1,800

2,000

-200

-10

Падение

6

2024-03-01

1,600

1,800

-200

-11.1111111111

Падение

7

2024-01-01

500

[NULL]

[NULL]

[NULL]

Нет данных

8

2024-03-01

800

500

300

60

Рост

9

2024-01-01

3,000

[NULL]

[NULL]

[NULL]

Нет данных

10

2024-02-01

4,500

3,000

1,500

50

Рост

11

2024-03-01

2,500

4,500

-2,000

-44.4444444444

Падение

Задача №8.

Задача: Заполнить таблицу иерархии отделов рекурсивно, начиная с родительского отдела.

Таблица:

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    parent_dept_id INT,  
    dept_name VARCHAR(100)  
);
```

Решение

The screenshot shows a database client interface with a sidebar on the left displaying a database schema. The main window shows a SQL query and its results.

SQL Query:

```

-- Задание 8
WITH RECURSIVE dept_tree AS (
  -- Базовый запрос
  -- Формирует родительский уровень
  SELECT
    dept_id,
    parent_dept_id,
    dept_name,
    1 AS level,
    dept_name::VARCHAR(1000) AS path
  FROM test_sber.departments
  WHERE parent_dept_id IS NULL
  UNION ALL
  -- Рекурсивная часть
  -- На каждой итерации обрабатываются только отделы найденные на предыдущем шаге
  -- Когда результатом JOIN получаем 0 строк, рекурсия останавливается
  -- Шаги происходят под капотом WITH RECURSIVE, функция хранит рабочий набор значений с предыдущего шага
  SELECT
    d.dept_id,
    d.parent_dept_id,
    d.dept_name,
    dt.level + 1,
    (dt.path || ' ' || d.dept_name)::VARCHAR(1000)
  FROM test_sber.departments AS d
  INNER JOIN dept_tree AS dt ON d.parent_dept_id = dt.dept_id
)
SELECT
  dept_id,
  parent_dept_id,
  dept_name,
  level,
  path
FROM dept_tree
ORDER BY path;

```

Results 1

id	dept_id	parent_dept_id	dept_name	level	path
1	1	[NULL]	Головной офис	1	Головной офис
2	12	1	Бухгалтерия	2	Головной офис -> Бухгалтерия
3	122	12	Казначейство	3	Головной офис -> Бухгалтерия -> Казначейство
4	121	12	Налоговый отдел	3	Головной офис -> Бухгалтерия -> Налоговый отдел
5	120	12	Расчетный отдел	3	Головной офис -> Бухгалтерия -> Расчетный отдел
6	10	1	ИТ-департамент	2	Головной офис -> ИТ-департамент
7	100	10	Разработка ПО	3	Головной офис -> ИТ-департамент -> Разработка ПО
8	101	10	Системное администрирование	3	Головной офис -> ИТ-департамент -> Системное администрирование
9	102	10	Техподдержка	3	Головной офис -> ИТ-департамент -> Техподдержка
10	11	1	Отдел продаж	2	Головной офис -> Отдел продаж
11	110	11	Активные продажи	3	Головной офис -> Отдел продаж -> Активные продажи
12	111	11	Работа с клиентами	3	Головной офис -> Отдел продаж -> Работа с клиентами
13	112	11	Тендерный отдел	3	Головной офис -> Отдел продаж -> Тендерный отдел

Задача №9.

Задача: Получить список пользователей, которые ни разу не делали заказы, вместе с информацией о последнем заказе каждого пользователя.

Таблицы:

```

CREATE TABLE users (
  user_id INT PRIMARY KEY,
  username VARCHAR(100)
);

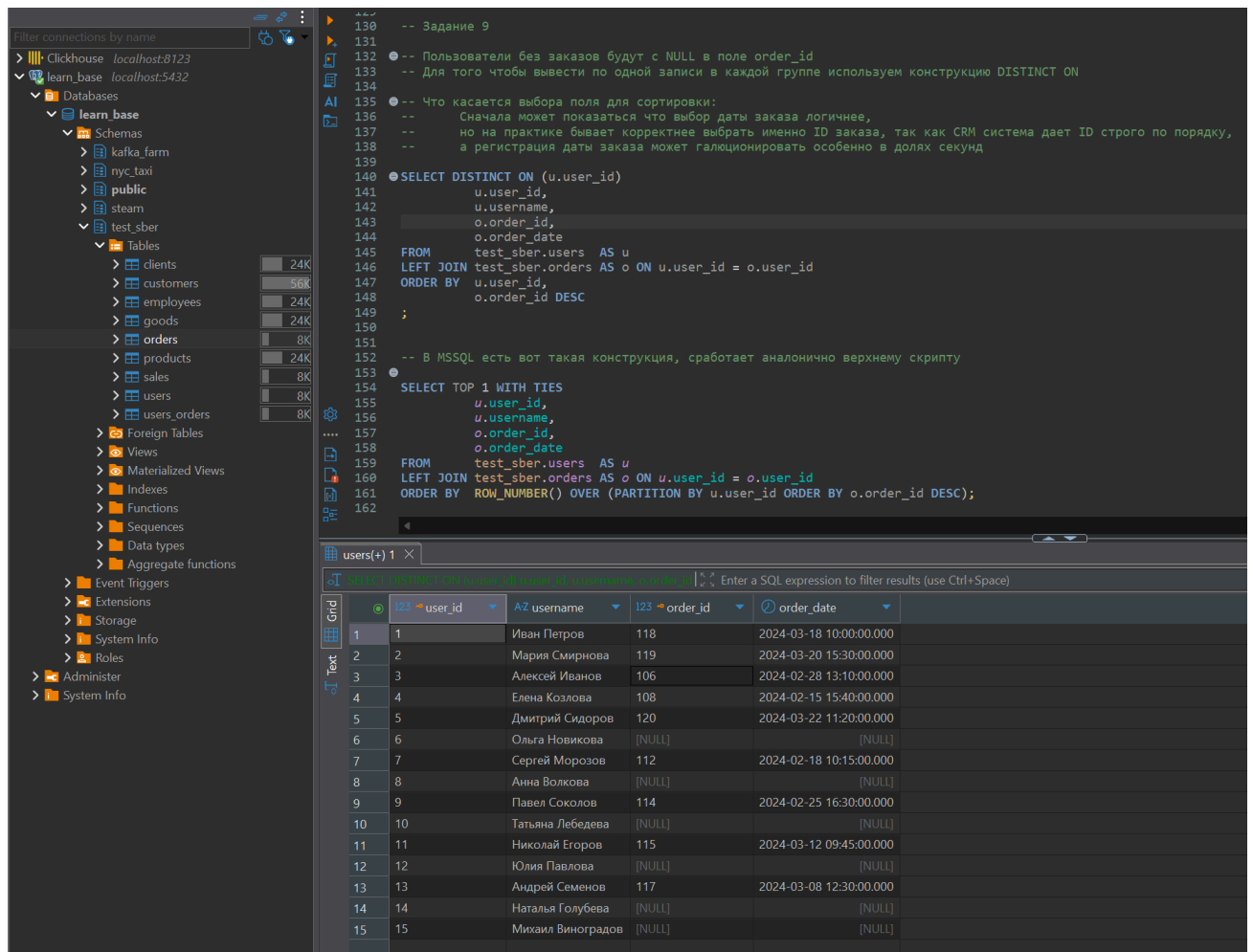
```

```

CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  user_id INT,
  order_date TIMESTAMP
);

```

Решение



Задача №10.

Задача: Объяснить план выполнения следующего запроса:

```

SELECT
    p.product_id,
    SUM(s.revenue) AS total_revenue
FROM products p
JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id
ORDER BY total_revenue DESC
LIMIT 10;

```

Опишите структуру плана выполнения и предложите оптимизацию, если применимо.

Решение


```

206
207 -- Задание 10
208
209 ● EXPLAIN
210 SELECT
211     p.product_id,
212     SUM(s.revenue) AS total_revenue
213 FROM   test_sber.products AS p
214 JOIN   test_sber.sales AS s ON p.product_id = s.product_id
215 GROUP BY p.product_id
216 ORDER BY total_revenue DESC
217 LIMIT 10
218
Results 1
EXPLAIN SELECT p.product_id, SUM(s.revenue) AS total_revenue
AZ QUERY PLAN
1 Limit (cost=238.94..238.97 rows=10 width=36)
2 -> Sort (cost=238.94..238.99 rows=20 width=36)
3   Sort Key: (sum(s.revenue)) DESC
4   -> HashAggregate (cost=238.26..238.51 rows=20 width=36)
5     Group Key: p.product_id
6     -> Hash Join (cost=1.45..188.26 rows=10000 width=10)
7       Hash Cond: (s.product_id = p.product_id)
8       -> Seq Scan on sales s (cost=0.00..155.00 rows=10000 width=10)
9       -> Hash (cost=1.20..1.20 rows=20 width=4)
10        -> Seq Scan on products p (cost=0.00..1.20 rows=20 width=4)
  
```

```

Limit (cost=238.94..238.97 rows=10 width=36) -- Ограничение строк
-> Sort (cost=238.94..238.99 rows=20 width=36) -- Сортировка результата
Sort Key: (sum(s.revenue)) DESC -- условия сортировки
-> HashAggregate (cost=238.26..238.51 rows=20 width=36) -- Агрегация с использованием хэширования
Group Key: p.product_id -- группировка по полю product_id
-> Hash Join (cost=1.45..188.26 rows=10000 width=10) -- Соединение таблиц через хэш таблицу
Hash Cond: (s.product_id = p.product_id) -- по полю product_id
-> Seq Scan on sales s (cost=0.00..155.00 rows=10000 width=10) -- Сканирование таблицы sales (10к строк)
-> Hash (cost=1.20..1.20 rows=20 width=4) -- Создание хэш таблицы в памяти
-> Seq Scan on products p (cost=0.00..1.20 rows=20 width=4) -- Сканирование таблицы products (20 строк)
  
```

Основные моменты

- Планировщик проигнорировал индексы на таблицах (хотя они были) и использовал полное сканирование. Ну, ему видимо виднее. Решил что фуллскан будет быстрее.
- Самые дорогие операции:
 - сканирование sales (10 тыс строк): 155
 - соединение таблиц: 33
 - агрегация: 50

Предложения по улучшению:

- Основная проблема, что мы джойним огромную простыню к маленькой таблице.
- Соответственно, если сначала выполним агрегацию, а потом соединим две маленькие таблицы, получим прирост скорости за счет этой операции

Проверим

The screenshot shows the ClickHouse GUI interface. On the left, a tree view displays the database structure: Clickhouse (localhost:8123) and learn_base (localhost:5432). Under learn_base, there are Schemas (kafka_farm, nyc_taxi, public, steam) and Tables (clients, customers, departments, employees, goods, orders, products, sales, users, users_orders). The 'sales' table is highlighted with a size of 1.3M.

The main query editor shows the following SQL query:

```

220
221 EXPLAIN
222 WITH agg AS (
223     SELECT s.product_id, SUM(s.revenue) AS total_revenue
224     FROM test_sber.sales AS s
225     GROUP BY s.product_id
226 )
227 SELECT
228     p.product_id,
229     a.total_revenue
230 FROM test_sber.products AS p
231 JOIN agg AS a ON p.product_id = a.product_id
232 ORDER BY total_revenue DESC
233 LIMIT 10
234

```

The 'Results' tab shows the execution plan (AZ QUERY PLAN) with the following steps:

Step	Operation	Cost	Rows	Width
1	Limit	(cost=207.40..207.42)	rows=10	width=36
2	-> Sort	(cost=207.40..207.45)	rows=20	width=36
3	Sort Key: a.total_revenue DESC			
4	-> Hash Join	(cost=205.70..206.96)	rows=20	width=36
5	Hash Cond: (p.product_id = a.product_id)			
6	-> Seq Scan on products p	(cost=0.00..1.20)	rows=20	width=4
7	-> Hash	(cost=205.45..205.45)	rows=20	width=36
8	-> Subquery Scan on a	(cost=205.00..205.45)	rows=20	width=36
9	-> HashAggregate	(cost=205.00..205.25)	rows=20	width=36
10	Group Key: s.product_id			
11	-> Seq Scan on sales s	(cost=0.00..155.00)	rows=10000	width=10

```

Limit (cost=207.40..207.42 rows=10 width=36)
-> Sort (cost=207.40..207.45 rows=20 width=36)
Sort Key: a.total_revenue DESC
-> Hash Join (cost=205.70..206.96 rows=20 width=36) -- А вот и наш профит, JOIN упал с 30 до 1
Hash Cond: (p.product_id = a.product_id)
-> Seq Scan on products p (cost=0.00..1.20 rows=20 width=4)
-> Hash (cost=205.45..205.45 rows=20 width=36)
-> Subquery Scan on a (cost=205.00..205.45 rows=20 width=36)
-> HashAggregate (cost=205.00..205.25 rows=20 width=36) -- Агрегация раньше, те же 50
Group Key: s.product_id
-> Seq Scan on sales s (cost=0.00..155.00 rows=10000 width=10) -- Фулл скан, те же 155

```

Как видим теория подтвердилась